

Hệ thống đọc hiểu và trả lời câu hỏi tiếng Việt

Vietnamese Extractive Machine Reading Comprehension
trên UIT-ViQuAD 2.0

Bốn mô hình · 500 câu kiểm định · mọi số đo sinh từ results/

NỘI DUNG — SÁU MỤC

1 · Tổng quan

2 · Liên quan

3 · Dữ liệu

4 · Phương pháp

5 · Kết quả

6 · Kết luận

NHÓM 7

Lê Quang Thi
25210337

Trần Trọng Tấn
25210334

Nguyễn Quang Lâm
25210289

Vỏ Cẩm Thu
25210342

GVHD

ThS. Nguyễn Hữu Quyền

Bối cảnh & động lực — vì sao bài toán này quan trọng

Tìm kiếm trả về TÀI LIỆU. Đọc hiểu máy trả về CÂU TRẢ LỜI.

Khối lượng văn bản một người phải đọc để tìm một thông tin cụ thể tăng nhanh hơn thời gian họ có. Công cụ tìm kiếm giải quyết một nửa: trả về tài liệu liên quan. MRC giải quyết nửa còn lại — chỉ thẳng vào đoạn chữ trả lời câu hỏi.

BA CHỖ DỪNG TRỰC TIẾP

- Trợ lý học tập: hỏi trên giáo trình, quy chế, tài liệu môn học
- Hỏi đáp trên tài liệu nội bộ của doanh nghiệp
- Tầng “đọc” trong pipeline truy hồi rồi đọc (retrieve-then-read)

BỐN KHÓ KHĂN RIÊNG CỦA TIẾNG VIỆT

- 1 Đơn vị chữ viết là âm tiết, không phải từ**
“Hà Nội”, “thủ đô” là từ gồm hai âm tiết cách nhau bởi khoảng trắng
- 2 Dấu thanh mang nghĩa**
“hoà” ≠ “hoa”; một bước chuẩn hoá làm mất dấu là sai EM mà không báo lỗi
- 3 Không có mạo từ**
quy tắc bỏ a/an/the của SQuAD không áp được: “các”, “những” có thể thuộc đáp án
- 4 Khoảng một phần ba câu hỏi không có đáp án**
hệ thống phải biết TỪ CHỐI, không chỉ biết tìm

Hướng dẫn môn học: “Kết quả cao trên benchmark không đồng nghĩa với khả năng triển khai thực tế.”

Nên mục tiêu của đồ án không phải điểm cao nhất, mà: mọi con số được trình bày đều đúng, truy vết được, và được diễn giải đúng phạm vi thực nghiệm.

Phát biểu bài toán — trích xuất, không phải sinh

ĐỊNH NGHĨA HÌNH THỨC

Cho đoạn văn C là chuỗi ký tự và câu hỏi Q , tìm cặp chỉ số ký tự (s , e):

$0 \leq s < e \leq |C|$ sao cho $C[s:e]$ trả lời Q

hoặc trả về chuỗi rỗng khi C không chứa câu trả lời.

VÍ DỤ

Hà Nội là thủ đô của Việt Nam. Hà Nội được UNESCO công nhận là Thành phố vì hoà bình vào **năm 1999**.

CÂU HỎI

Hà Nội được UNESCO công nhận vào năm nào?

Đáp án: năm 1999

Mô hình không sinh chữ mới — nó chỉ ra vị trí.

ĐẶC TẢ ĐẦU VÀO / ĐẦU RA

ĐẦU VÀO

context: đoạn văn Wikipedia tiếng Việt · question: câu hỏi tự nhiên

ĐẦU RA

answer: một chuỗi CON của context, hoặc chuỗi rỗng khi không có đáp án

LOẠI BÀI TOÁN

trích xuất (extractive) — không sinh văn bản

BỘ DỮ LIỆU

UIT-ViQuAD 2.0, định dạng SQuAD-2.0

THANG ĐO

Exact Match và F1 trên token (âm tiết)

$\text{predict}(\text{ctx}, q) \subset \text{ctx}$

Tính trích xuất không phải chi tiết phụ — nó cho một bất biến kiểm được bằng máy.

Với mọi mô hình và mọi đầu vào, đầu ra phải là chuỗi con của context. Bất biến này được test tự động và bắt được cả lỗi lệch offset của tokenizer lẫn trường hợp mô hình “bịa” chữ không có trong văn bản.

Đóng góp của nhóm

1 Pipeline MRC tiếng Việt hoàn chỉnh

Từ nạp dữ liệu đến ứng dụng web, tổ chức thành module có kiểm thử — 474 test, 20 tệp.

2 So sánh có kiểm soát bốn mô hình

Mỗi bước cô lập đúng một biến: so khớp từ khoá → chuyển giao đa ngữ zero-shot → fine-tune trong miền → pretraining chuyên tiếng Việt.

3 Tách answerable / impossible

Cho thấy điểm tổng che khuất hai kỹ năng tách biệt — tìm đúng biên đáp án, và biết khi nào không nên trả lời.

4 Chặn bốn lỗi “sai âm thầm”

Test split không có nhãn · giới hạn số cửa sổ của transformers · mẫu đánh giá thiên lệch chủ đề · ngưỡng độ dài đáp án phụ thuộc tokenizer.

Và một đóng góp về phương pháp: chặn đoán một ca lỗi thay vì che nó đi.

Thất bại của ViSoBERT được phân tích theo quy trình loại trừ, gắn với bằng chứng đo được về mức độ phân mảnh token — trình bày ở mục 5.

Bốn hướng tiếp cận và các bộ dữ liệu đã có

KHẢO SÁT HƯỚNG TIẾP CẬN

- **So khớp từ khoá / truy hồi**
TF-IDF + cosine trên câu — không huấn luyện, dùng làm sàn
- **Chuyển giao đa ngữ zero-shot**
encoder đa ngữ đã fine-tune QA tiếng Anh, áp thẳng sang tiếng Việt (XLM-R squad2)
- **Fine-tune trong miền, encoder đa ngữ**
mBERT + lớp QA, huấn luyện trên chính dữ liệu tiếng Việt
- **PLM đơn ngữ tiếng Việt**
PhoBERT (2020) và ViSoBERT (2023) — hướng được kỳ vọng nhất cho tiếng Việt

CÁC BỘ DỮ LIỆU ĐÃ CÓ

BỘ DỮ LIỆU	QUY MÔ	KHÔNG ĐÁP ÁN
SQuAD 1.1 · 2016	> 100.000 câu (EN)	không
SQuAD 2.0 · 2018	+ > 50.000 câu	có
UIT-ViQuAD · 2020	> 23.000 câu · 174 bài (VI)	không
UIT-ViQuAD 2.0 · 2021	28.454 / 3.814 / 7.301	có

Dòng cuối là số đo trực tiếp từ tập tải về (train / validation / test) — không chép từ tài liệu; chi tiết ở mục 3.

UIT-ViQuAD 2.0 là bộ dữ liệu tiếng Việt duy nhất trong nhóm này có câu hỏi không có đáp án — nên nó là lựa chọn bắt buộc nếu muốn đo kỹ năng từ chối.

Bốn mô hình của đồ án chính là bốn hướng tiếp cận trên — đo trên cùng một tập, cùng một interface.

Nhờ vậy chênh lệch giữa hai dòng bất kỳ trong bảng kết quả đọc được thành “biến nào đã thay đổi”, thay vì chỉ là hai con số cạnh nhau.

GAP — chỗ nhóm đứng vào

1

Điểm tổng che hai kỹ năng khác nhau

Công bố theo bảng xếp hạng thường báo EM/F1 TỔNG. Với khoảng 30% câu không có đáp án, số tổng trộn “tìm đúng span” với “biết từ chối”.

NHÓM LÀM GÌ

Nhóm tách answerable_only / impossible_only cho cả bốn mô hình, kèm khoảng tin cậy.

2

Test split công khai đã bị lược nhỡ

Cả 7.301 câu test có gold rỗng nhưng is_impossible = False. Ai chấm trên đó sẽ thấy một mô hình luôn trả rỗng đạt EM 100%.

NHÓM LÀM GÌ

Nhóm đo, nói rõ, và chặn bằng assert_gradeable() trước mọi lần chấm.

3

Ngân sách cửa sổ phụ thuộc tokenizer

Khi đổi sang encoder tiếng Việt, max_length và độ dài đáp án tối đa thường được giữ nguyên theo tokenizer cũ — một biến gây nhiễu ít được nói tới.

NHÓM LÀM GÌ

Nhóm đo mức phân mảnh token và coi đó là confound, không phải kết luận về encoder.

GAP nhóm đứng vào là tính ĐÚNG ĐẴN của đánh giá, không phải điểm số.

Đề tài không yêu cầu vượt bảng xếp hạng. Giá trị của đồ án nằm ở chỗ mỗi con số chịu được chất vấn: biết nó đo cái gì, trên bao nhiêu mẫu, với độ bất định bao nhiêu.

UIT-ViQuAD 2.0 — thống kê đo trực tiếp từ tệp tải về

TAIDNG/UIT-VIQUAD2.0 → BA TẬP JSON ĐỊNH DẠNG SQUAD-2.0

SPLIT	CÂU HỎI	CONTEXT	ARTICLE	IMPOSSIBLE	CHẤM ĐƯỢC
train	28.454	4.101	138	9.216	28.454
validation	3.814	557	19	1.161	3.814
test	7.301	1.241	48	0	0

Đơn vị dedup và chống rò rỉ là CONTEXT, không phải ARTICLE: train có 138 bài viết nhưng 4.101 đoạn văn — nhằm hai đơn vị này làm vô hiệu hoá chính cơ chế bảo vệ.

PHÁT HIỆN: TEST SPLIT LÀ TẬP ẨN NHÃN

7.301 câu test đều có gold rỗng, đồng thời `is_impossible = False`.

Hai điều đó mâu thuẫn nhau nếu coi là nhãn thật ⇒ đáp án đã bị lược bỏ để dùng cho bảng xếp hạng.

Chấm nhầm trên tập này → EM 100%.

⇒ Mọi kết quả của đồ án đo trên validation.

Chặn bằng code, không bằng lời hứa
`assert_gradeable(examples)`

0

đoạn văn chung giữa train và validation — `assert_no_leakage()`
 xác nhận trước mỗi lần fine-tune

449 / 133

đoạn văn test chung với train / với validation — cảnh báo cho ai định tự gán nhãn lại test

Hệ quả thiết kế: validation là tập đánh giá cuối, và điều đó được nói rõ trong phần hạn chế.

Validation cũng được dùng để chọn epoch (trên mẫu 300 câu), nên hai mô hình fine-tune có thể lạc quan nhẹ so với một tập kiểm tra độc lập hoàn toàn.

Phân bố nhãn và phân bố độ dài

PHÂN BỐ NHÃN

TẬP	KHÔNG CÓ ĐÁP ÁN	TỈ LỆ
train	9.216 / 28.454	32,39%
validation	1.161 / 3.814	30,44%
test	— (đã lược nhãn)	—
mẫu đánh giá 500 câu	139 / 500	27,80%

Điểm tổng là trung bình có trọng số của hai kỹ năng theo đúng tỉ lệ này — mục 5 bóc tách ra.

PHÂN BỐ ĐỘ DÀI (ĐƠN VỊ: ÂM TIẾT, TÁCH THEO KHOẢNG TRẮNG)

CHỈ SỐ	TRAIN	VALIDATION
Đoạn văn — TB / trung vị / max	179,0 / 160 / 1.537	167,6 / 152 / 618
Đáp án vàng — TB / trung vị / p95	9,95 / 6 / 31	9,81 / 6 / 30
Câu hỏi — TB	14,6	14,2
Đoạn văn dài hơn 300 từ	239 / 4.101 · 5,8%	—

Đáp án có trung vị 6 âm tiết nhưng p95 = 31: đuôi dài, và đuôi này quyết định ngưỡng độ dài đáp án theo token.

MẪU ĐÁNH GIÁ 500 CÂU THEO NHÓM ĐỘ DÀI ĐOẠN VĂN — NGUỒN: RESULTS/EVAL_*.JSON

1 câu

< 100 từ · n < 30 ⇒ cỡ unreliable

405 câu

100 – 200 từ

85 câu

200 – 300 từ

9 câu

300+ từ · n < 30 ⇒ cỡ unreliable

Hai hệ quả đọc thẳng ra từ phân bố, không phải suy diễn.

Một: mẫu 500 câu gần như không có đoạn dài (9 câu ở nhóm 300+), nên câu hỏi “độ dài context ảnh hưởng thế nào” CHƯA được trả lời — nhóm ghi vào phần hạn chế. Hai: đuôi dài của đáp án là lý do L_max phải đặt theo tokenizer.

Phân bố chủ đề — và cái bẫy lấy mẫu nó gây ra

VALIDATION: 19 BÀI VIẾT · 3.814 CÂU HỎI

BÀI VIẾT (TOP 5)	SỐ CÂU	%
Franklin D. Roosevelt	503	13,2
Paris	462	12,1
Chiến tranh Vùng Vịnh	379	9,9
Canada	356	9,3
Mahatma Gandhi	324	8,5

5 bài lớn nhất chiếm 53,1% · đuôi rất mỏng: Montréal 42 · Kiến 57 · Gia cầm 59 câu. Chỉ 3 trong 19 bài viết thuộc khoa học tự nhiên (169 câu, 4,4%) — còn lại là lịch sử, chính trị, địa lý.

CÁI BÃY: CÂU HỎI TRONG TẬP ĐƯỢC SẮP THEO BÀI VIẾT
300 câu đầu tập validation thuộc DUY NHẤT một bài viết: “Paris”.

42,00 so với **52,00**

EM của cùng một checkpoint mBERT (epoch 2): 300 câu đầu tập so với 300 câu ngẫu nhiên seed 42. Chênh 10 điểm chỉ do CÁCH LẤY MẪU, không do mô hình.

`reproducible_subset(examples, n, seed=42)`

PHÂN BỐ TỪ ĐỂ HỎI (PHÂN NHÓM TỪ KHOÁ THÔ, CHỈ ĐỂ MÔ TẢ)

30,3%

gì

27,6%

nào

9,1%

ai

8,3%

như thế nào

7,9%

bao nhiêu

5,1%

vì sao + tại sao

Vì phân bố chủ đề lệch, mẫu đánh giá phải ngẫu nhiên có seed — và mẫu đó được kiểm lại.

Mẫu 500 câu phủ 19/19 bài viết, 5 bài lớn nhất chiếm 57,0% so với 53,1% của toàn tập: mẫu đại diện được về chủ đề. Cùng hàm lấy mẫu dùng cho cả đường cong huấn luyện và bảng kết quả cuối, nên hai nguồn số liệu so sánh được với nhau.

Bốn hệ thống — mỗi cái trả lời một câu hỏi khác nhau

1 TF-IDF Baseline

không huấn luyện

Bài toán này giải được bằng so khớp từ khoá thuần không?

2 XLM-R squad2

zero-shot

Transfer từ SQuAD-2.0 tiếng Anh sang tiếng Việt được bao nhiêu?

3 mBERT + QA

fine-tuned · MPS

Fine-tune trong miền thêm được bao nhiêu, với encoder đa ngữ?

4 ViSoBERT + QA

fine-tuned · MPS

Pretraining chuyên tiếng Việt đóng góp bao nhiêu?

Giả thuyết được đăng ký TRƯỚC khi chạy (results/hypotheses.md) — và nó viết cho PhoBERT.

Giả thuyết dự đoán encoder tiếng Việt sẽ vượt mBERT. PhoBERT không chạy được vì lý do kỹ thuật (slide sau), nên ViSoBERT là mô hình THAY THẾ, không phải phép kiểm cho giả thuyết đó: giả thuyết trung tâm vẫn CHƯA kiểm được, không phải bị bác bỏ.

Bốn tầng, một chiều phụ thuộc

Tầng dữ liệu

data.py · tagging.py

parse SQuAD-2.0 → Example · dedup & split theo CONTEXT ·
assert_no_leakage · assert_gradeable

Tầng mô hình

predictor.py · baseline_tfidf.py · transformer_qa.py ·
windowing.py · features.py · training.py

một interface duy nhất: Predictor.predict(ctx, q) -> str

Tầng đánh giá

normalize.py · metrics.py · evaluate.py

EM + token-F1 · tách answerable / impossible · provenance · cờ unreliable khi n
< 30

Tầng trình bày

src/demo/ (hàm thuần) → app/ (chỉ gọi widget) ·
make_figures.py · make_report.py

CHỈ ĐỌC results/ — không bao giờ sinh ra con số

app/ → src/demo/ → src/mrc/ → results/, data/raw/ · một mũi tên, một chiều, không ngoại lệ

Bốn bất biến — và cơ chế thực thi bằng máy

1

Extractive

Mọi `predict()` trả về chuỗi con của context

CƠ CHẾ THỰC THI

Test cho mọi Predictor; `decode_span()` ném `ValueError` nếu khoảng không hợp lệ

2

Không leakage

Một context chỉ thuộc một split

CƠ CHẾ THỰC THI

`assert_no_leakage()` chạy trong mọi đường split và làm FAIL cả run

3

Truy vết được

Mọi kết quả mang `commit`, `timestamp`, `device`, `split`, `n`

CƠ CHẾ THỰC THI

`run_evaluation()` sinh các trường này — không đường nào tạo ra kết quả thiếu chúng

4

Có n kèm số

Nhóm `count < 30` tự gắn `unreliable: true`

CƠ CHẾ THỰC THI

`breakdown()` gán cờ; không bảng nào trong báo cáo có số thiếu `n`

Bất biến thứ năm: demo không sinh ra con số nào — và điều đó do kernel bắt buộc, vì `models/`, `data/`, `results/` được mount read-only.

Thước đo được cài trước vật được đo

HAI THANG ĐO, HAI CÂU HỎI KHÁC NHAU

EM — đúng 1 điểm khi chuỗi dự đoán sau chuẩn hoá TRÙNG KHÍT một đáp án vàng; lệch một ký tự là 0.

F1 — trên TOKEN (âm tiết), dùng Counter (đa tập) thay vì set để token lặp được đếm đúng; cho điểm bán phần.

Câu không có đáp án — đúng khi và chỉ khi trả về chuỗi rỗng.
Nhiều đáp án vàng: lấy điểm cao nhất.

Bằng chứng: TF-IDF có F1 23,09 nhưng EM chỉ 0,80 — nó trả về cả một câu: trùng token, gần như không bao giờ trùng khít.

BA QUYẾT ĐỊNH CHUẨN HOÁ — MỖI CÁI ĐƯỢC TEST GHIM

- A Không loại bỏ mạo từ** “các tỉnh” giữ nguyên “các”
tiếng Việt không có mạo từ; “các”, “những”, “con” là loại từ và có thể thuộc đáp án
- B Tách token theo khoảng trắng** “Hà Nội” là 2 token
khớp cách đánh giá thông dụng của ViQuAD; không nhiễm lỗi bộ tách từ
- C Giữ dấu tiếng Việt** $EM(\text{“hoà”, “hoa”}) = 0$
“hoà” $\neq$ “hoa”; bỏ dấu làm sai EM hàng loạt mà không báo lỗi

Hai chi tiết cài đặt cũng được test:

dấu câu nhận diện theo nhóm Unicode P* thay vì string.punctuation (bắt được “—”, “”, “...” thường gặp trong Wikipedia) · evaluate() raise KeyError nếu thiếu dự đoán của bất kỳ câu nào — bỏ qua âm thầm sẽ tính EM trên một tập con nhỏ hơn mà không ai biết.

Metric được cài trước mọi mô hình, và đạt độ phủ kiểm thử 100%.

Thứ tự này có chủ ý: nếu viết mô hình trước rồi mới viết thước đo, mọi lần “sửa metric cho hợp lý” đều là một cơ hội để chỉnh số theo ý mình mà không nhận ra.

Hai quyết định kỹ thuật đã đổi cả hệ thống

ADR-003

Tự cài cửa sổ trượt

return_overflowing_tokens của transformers 5.17.0 sinh tối đa 2 cửa sổ, bất kể context dài bao nhiêu:

ĐỘ DÀI CONTEXT	THỰC TẾ	ĐÚNG RA
210 token	2	2
420 token	2	5
700 token	2	8
1400 token	2	16

Đuôi context bị cắt ÂM THẨM: không exception, không cảnh báo — đáp án nằm cuối đoạn văn thì không bao giờ tìm được.

ADR-004 · ADR-005

Dấu tiếng Việt là ràng buộc cứng

PhoBERT không có fast tokenizer ⇒ không có offset_mapping ⇒ cách duy nhất lấy lại chuỗi là tokenizer.decode().

decode() làm mất dấu: “hoà” → “hoa”

Sai một dấu là sai cả EM lẫn bất biến substring. Nên hệ thống cắt thẳng chuỗi gốc theo offset ký tự — và TransformerQA TỪ CHỐI khởi tạo mô hình không có fast tokenizer.

⇒ Thay PhoBERT bằng uitnlp/visobert — cũng là encoder tiếng Việt, của chính UIT NLP

Huấn luyện: nhãn ký tự → token, chọn epoch bằng hàm thuần

SIÊU THAM SỐ — ĐỌC TỪ RESULTS/TRAINING_CURVE_*.JSON

THAM SỐ	MBERT	VISOBERT
Số epoch · seed	2 · 42	3 · 42
Batch × tích lũy gradient	12 × 2	12 × 2
Tốc độ học	3·10 ⁻⁵	5·10 ⁻⁵
max_length / doc_stride	384 / 128	384 / 128
L_max (độ dài đáp án tối đa)	30	64
Số cửa sổ huấn luyện	30.540	40.984

CHỖ SAI ÂM THẨM NGUY HIỂM NHẤT CỦA CẢ PIPELINE

Chuyển vị trí đáp án từ KÝ TỰ sang TOKEN. Nếu ánh xạ lệch, mô hình học nhãn sai mà loss vẫn giảm bình thường.

⇒ Bất biến được test không phải “chạy không lỗi”, mà: giải mã ngược nhãn token phải ra ĐÚNG chuỗi đáp án vàng.

CHỌN EPOCH

Theo F1 validation, không theo loss (loss là đại lượng trên train). Logic nằm trong training.py dưới dạng hàm thuần — test trong vài mili-giây thay vì chờ một epoch 35 phút.

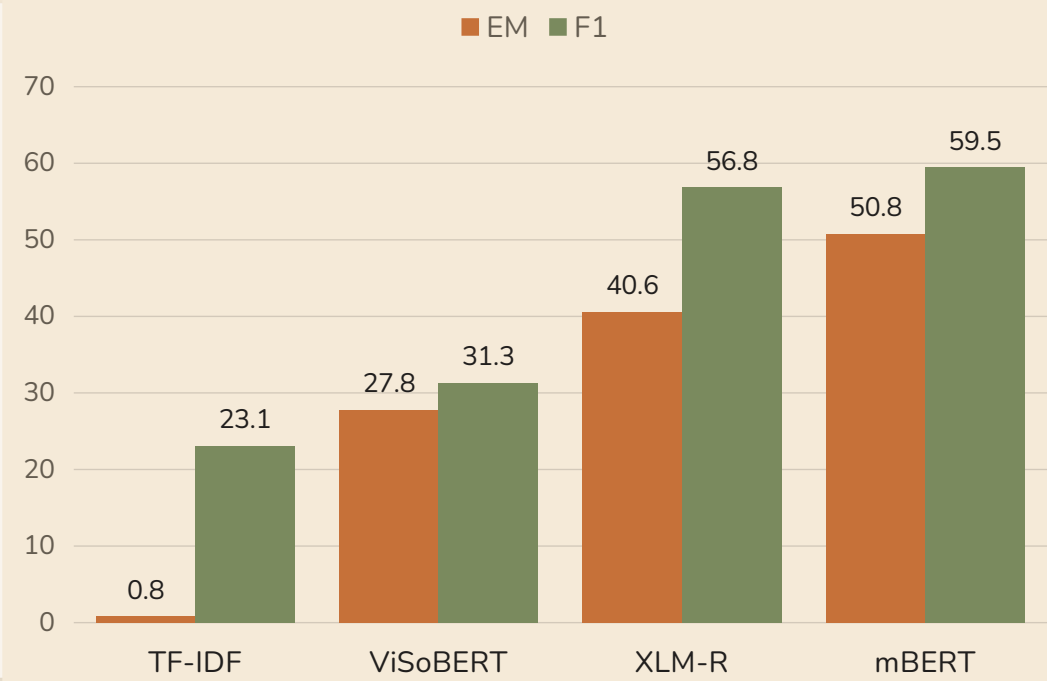
ĐƯỜNG CONG HUẤN LUYỆN — CHẨN ĐOÁN TỰ ĐỘNG, NGƯỠNG COLLAPSE_TOLERANCE = 0,5

MÔ HÌNH · EPOCH	LOSS TRAIN	VAL EM	VAL F1	F1 – EM	CHẨN ĐOÁN
mBERT · epoch 1	2,1316	46,00	58,39	12,39	còn cải thiện
mBERT · epoch 2	1,2960	52,00	59,75	7,75	epoch được chọn làm mô hình cuối
ViSoBERT · epoch 1	3,0512	25,67	25,67	0,00	EM ≈ F1 ⇒ suy sụp về trả rỗng
ViSoBERT · epoch 2	2,5678	25,33	25,61	0,28	EM ≈ F1 ⇒ suy sụp về trả rỗng
ViSoBERT · epoch 3	2,1908	27,00	30,48	3,48	epoch được chọn làm mô hình cuối

F1 cho điểm bán phần nên bình thường phải cao hơn EM. Hai giá trị trùng nhau ⇒ mỗi câu chỉ có thể đúng hoàn toàn hoặc sai hoàn toàn — dấu hiệu suy sụp về “luôn trả rỗng”, và training.py tự nhận ra điều đó.

UIT-ViQuAD 2.0 · validation · n = 500 · thiết bị MPS

MODEL	EM	F1	ĐỘ TRỄ
TF-IDF Baseline	0,80	23,09	0,54 ms
ViSoBERT + QA (fine-tuned)	27,80	31,31	22,73 ms
XLM-R squad2 (zero-shot)	40,60	56,84	14,35 ms
mBERT + QA (fine-tuned)	50,80	59,49	12,26 ms



50,80

EM cao nhất — mBERT fine-tuned

±4,4

khoảng tin cậy 95% của EM ở n = 500

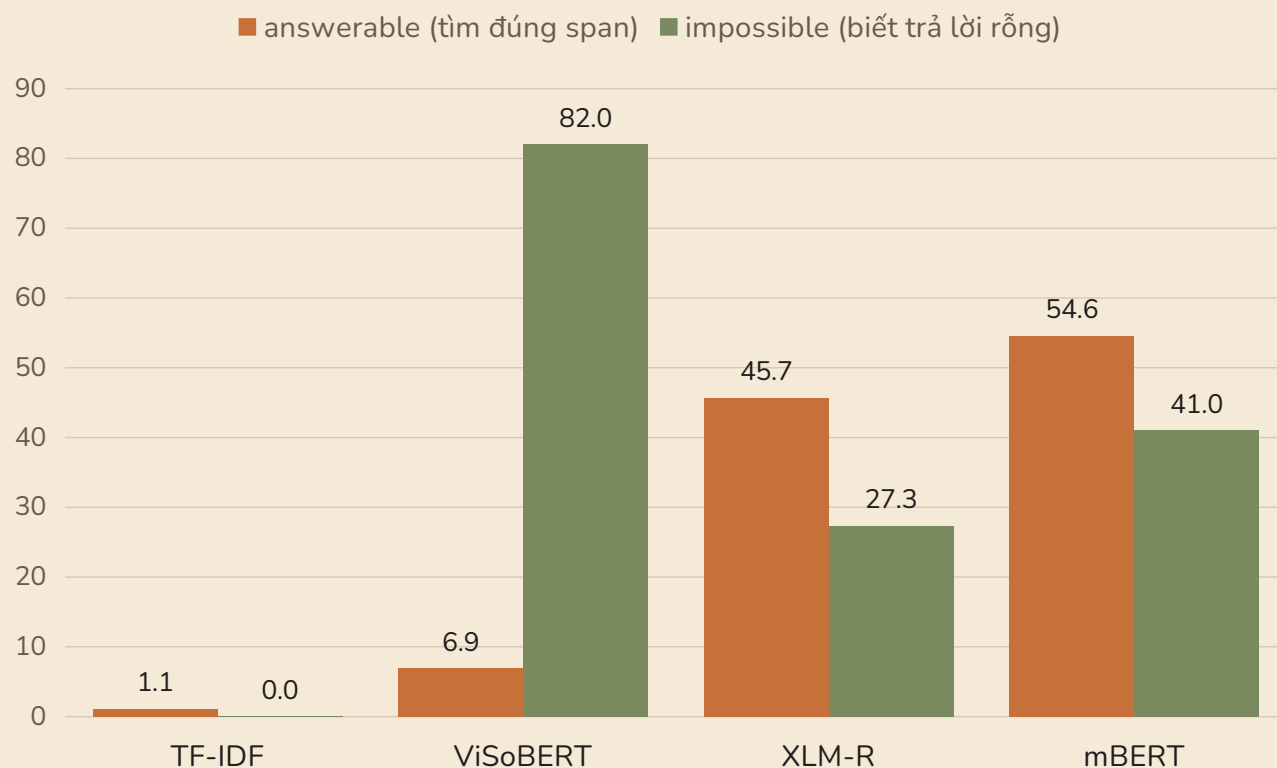
27,80%

câu không có đáp án trong mẫu

2

commit truy vết được cho 4 kết quả

mBERT thắng KHÔNG phải vì tìm span giỏi hơn



Trên câu answerable, XLM-R zero-shot thực ra TỐT HƠN

F1 68,20 so với 66,60

— nhưng chênh lệch 1,60 điểm này nằm gọn trong nhiễu, nên không phải kết luận vững.

mBERT thắng tổng thể vì biết khi nào KHÔNG nên trả lời

impossible EM 41,01 so với 27,34

+13,67 điểm $\pm$ 11,03 — có ý nghĩa thống kê. Với 27,80% câu trong mẫu là không có đáp án, chính kỹ năng thứ hai quyết định bảng xếp hạng.

Đây là lý do báo cáo tách answerable_only và impossible_only — con số tổng trộn hai kỹ năng và che mất điều này.

Vì sao ViSoBERT suy sụp — một ca lỗi được chẩn đoán

EM tổng 27,80

= tỉ lệ câu không có đáp án của mẫu 27,80%

Hai con số trùng nhau không phải trùng hợp — bóc tách ra:

82,01

impossible EM — gần như chỉ
ăn điểm từ việc từ chối

6,93

answerable EM — hầu như
không tìm được span nào

NGUYÊN NHÂN: TỪ VỰNG NHỎ CHIA VỤN VĂN BẢN WIKIPEDIA

	MBERT	VISOBERT
vocab	119.547	15.002
“Hà Nội là thủ đô...” → số token	17 token	23 token
context validation, TB (token)	204,6	326,5
context > 357 token	16 / 557	154 / 557

Mỗi cửa sổ thêm vào là một cơ hội nữa để điểm “rỗng” thắng.

BỐN YẾU TỐ ĐÃ ĐỦ ĐỂ GIẢI THÍCH SỰ SUY SỤP

max_length = 384 chưa chỉnh

CHƯA bù trừ — yếu tố mạnh nhất

loss chưa hội tụ

2,19 sau 3 epoch (mBERT 1,30 sau 2)

sức chứa nhỏ hơn 45%

97,0 M so với 177,3 M tham số

mất cân bằng lớp

32,39% impossible ⇒ hồ “luôn trả rỗng”

Nói rõ cho công bằng: L_max (độ dài ĐÁP ÁN) ĐÃ được bù, 30 → 64 — trực CHƯA bù là max_length. Hai tham số rất dễ nói lẫn.

Chưa kết luận được rằng tiền huấn luyện tiếng Việt không mang lại lợi ích.

Từ vựng nhỏ chia vụn văn bản Wikipedia và ngân sách cửa sổ không được chỉnh theo — lời giải thích PHÙ HỢP với bằng chứng, chưa phải nhân quả đã chứng minh. ViSoBERT chưa huấn luyện lại sau chẩn đoán, nên 27,80 EM phải đọc như MỘT LẦN HUẤN LUYỆN THẤT BẠI, không phải một phép đo năng lực encoder.

Ứng dụng: sáu màn hình, và một lệnh để chấm

● Hỏi đáp

/

nhập câu hỏi, kéo ngưỡng, xem span + bằng chứng

● Kết quả

/ket-qua

bảng EM/F1, breakdown, provenance

● Phân tích lỗi

/phan-tich-loi

pred (terracotta) so với gold (sage)

● Dữ liệu

/du-lieu

thống kê split, phân phối độ dài

● So sánh model

/so-sanh

bốn mô hình cạnh nhau, ô tốt nhất được TÍNH

● Huấn luyện

/huan-luyen

đường cong loss, EM, F1

KỸ THUẬT PHẦN MỀM — NGƯỜI CHẤM CHỈ CẦN DOCKER, MỘT ẢNH CHO CẢ TEST LẦN DEMO

```
docker compose run --rm tests
```

```
docker compose up app
```

```
./run.sh
```

423 test, ~1 giây, HF_HUB_OFFLINE=1

demo tại <http://localhost:8501>

nạp ảnh đã docker save rồi mở demo

474 test

bộ đầy đủ, 20 tệp (gồm
51 test tải mô hình thật)

2,49 GB

ảnh Docker (nén 501 MB)
— từ 10,2 GB

Demo không sinh ra con số nào.

Mọi số trên màn hình đọc từ results/*.json hoặc đo trực tiếp từ data/raw/ qua một điểm truy cập duy nhất — kể cả thứ hạng “tốt nhất” và ô in đậm trong bảng so sánh, chúng được TÍNH chứ không gõ tay. Huấn luyện lại là màn hình tự nói đúng.

Hạn chế — nói thẳng, và chỗ nào thì đi tiếp

— **PhoBERT chưa từng được chạy**

Giả thuyết trung tâm “encoder tiếng Việt vượt mBERT” CHƯA KIỂM ĐƯỢC — ViSoBERT là mô hình thay thế, không phải phép kiểm

— **ViSoBERT chưa huấn luyện lại sau chẩn đoán**

Lời giải thích max_length vẫn là giả thuyết phù hợp bằng chứng, chưa phải nhân quả

— **Đánh giá trên validation, không phải test**

Test là blind split — giới hạn của dữ liệu công khai, không phải của thiết kế. Validation cũng dùng để chọn epoch

— **n = 500, một seed duy nhất**

CI 95% khoảng $\pm 4,4$ điểm — chênh lệch F1 answerable giữa mBERT và XLM-R KHÔNG có ý nghĩa

— **Độ dài context và question_type**

Mẫu chỉ có 9 câu ở nhóm 300+ nên chưa trả lời được; question_type là heuristic tự gán, ngưỡng 0,6 chọn theo quan sát

HƯỚNG PHÁT TRIỂN

Huấn luyện lại ViSoBERT với max_length 768 và lr thấp hơn — phép kiểm trực tiếp · chạy --full trên 3.814 câu · thêm epoch cho mBERT (loss vẫn đang giảm) · tích hợp PhoBERT khi có đường lấy offset · mở rộng dữ liệu ra ngoài miền Wikipedia

KẾT LUẬN

Điểm tổng trộn hai kỹ năng: tìm span, và biết khi nào im lặng.

Còn câu hỏi “tiếng Việt chuyên biệt có giúp không?” thì vẫn để ngỏ — PhoBERT chưa chạy, lần huấn luyện ViSoBERT đã suy sụp vì một nguyên nhân cấu hình chưa được bù trừ. Nói rõ điều đó cũng là một kết quả.

50,80

EM · mBERT fine-tuned

4

mô hình, một interface

423

test trong ~1 giây

0

số đo được gõ tay — deck đọc
results/

Cảm ơn thầy và các bạn đã lắng nghe.

[docs/ARCHITECTURE.md](#) · [docs/SYSTEM_DESIGN.md](#)